

Reproducing “Bilinear MLPs Enable Weight-Based Mechanistic Interpretability”

Itay Erlich, May Ben Zion, Jacob Shashoua, Sagi Schwartz

FACT-AI Course Project
January 30, 2026

Bilinear MLPs: Weight-Based Interpretability

Purpose:

- ▶ Direct interpretability from weights
- ▶ No post-hoc analysis needed
- ▶ No SAE training required

Key Insight:

Bilinear structure creates **quadratic** interactions that can be **eigendecomposed** to reveal interpretable input directions.

Method:

- ▶ Replace nonlinearity with element-wise multiplication
- ▶ This creates an **interaction matrix** for each output
- ▶ Eigendecompose to find dominant input directions
- ▶ Top eigenvectors = most important patterns

Mathematical Foundation

Bilinear Layer: $\mathbf{y} = (\mathbf{W}_l \mathbf{x}) \odot (\mathbf{W}_r \mathbf{x})$

Output as Quadratic Form: For each output c : $y_c = \mathbf{x}^\top B_c \mathbf{x}$
where B_c is the **interaction matrix**—captures how input pairs (x_i, x_j) contribute to y_c .

Eigendecomposition: Symmetrize and decompose B_c :

$$B_c = \sum_{r=1}^d \lambda_r^{(c)} \mathbf{v}_r^{(c)} (\mathbf{v}_r^{(c)})^\top \quad \Rightarrow \quad y_c = \sum_r \lambda_r (\mathbf{v}_r^\top \mathbf{x})^2$$

Eigenvectors $\mathbf{v}_r^{(c)} \in \mathbb{R}^{d_{\text{in}}} =$ interpretable input directions; eigenvalues $\lambda_r^{(c)} =$ importance.

Effective Rank: Measures spectral concentration (how many eigenvalues matter):

$$\text{EffRank}(\boldsymbol{\lambda}) = \left(\frac{\|\boldsymbol{\lambda}\|_1}{\|\boldsymbol{\lambda}\|_2} \right)^2$$

Low effective rank $\Rightarrow$ few dominant eigenvectors $\Rightarrow$ more interpretable.

Two Analysis Methods (Paper Section 3)

The analysis method depends on what features are available:

Method A: No Prior Features

(Vision, Section 4)

Model: Single bilinear MLP + linear head (MNIST/Fashion-MNIST).

Classes act as output features.

Eigendecompose B_c directly:

$$y_c = \sum_r \lambda_r (\mathbf{v}_r^\top \mathbf{x})^2$$

Eigenvectors $\mathbf{v}_r$ visualized as 28×28 images.

Method B: SAE Features

(Language, Section 5)

Model: Transformer with bilinear MLPs (replacing standard MLPs) + pretrained SAEs.

SAEs $\rightarrow$ sparse features $\mathbf{z}^{\text{in}}, \mathbf{z}^{\text{out}}$.

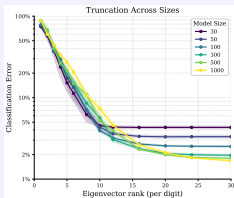
Build Q_f per output feature f :

$$\mathbf{z}_f^{\text{out}} \approx (\mathbf{z}^{\text{in}})^\top Q_f \mathbf{z}^{\text{in}}$$

Both methods use eigendecomposition to identify important input directions for each output.

Vision Results: Claims V1–V3

V1: Low-rank emergence ✓

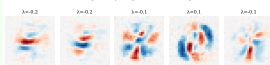


Top- k truncation preserves accuracy

V2: Interpretable eigenvectors ✓

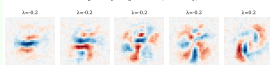
Full Reg (noise+WD):

Digit 0: Top-5 Eigenvectors (Full Reg)



Noise Only:

Digit 0: Top-5 Eigenvectors (Noise Only)



Both show clear “0” patterns

V3: Overfitting w/o reg ✓

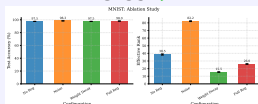


Digit 0, no reg: diffuse, noisy

Key insight: Regularization induces low-rank structure; **noise** makes eigenvectors interpretable.

Vision Results: Claims V4–V6

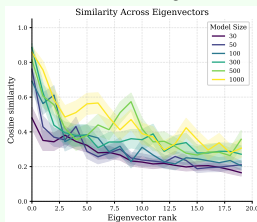
V4: Regularization roles ✓



WD ↓ rank; Noise ↑ acc

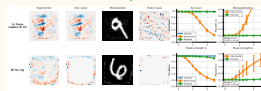
$$\text{EffRank ratio} = \frac{\text{EffRank(WD)}}{\text{EffRank(none)}} = \frac{15.5}{38.5} = 0.40 < 0.5 \quad \checkmark$$

V5: Cross-seed consistency ✓



Top eigenvectors highly similar across seeds; similarity decays with rank

V6: Adversarial gen. ✓



$\mathbf{m}_c = (V_c^+)_{:,1}^\dagger$: top positive eigenvector.

Pseudoinverse extracts V^+ from B_c .

Result: Eigenvector mask > random.

All 6 vision claims reproduced.

Language Results: Claims L1, L2, L4 (Figure 8)

Methodology:

1. **Data:** Run $\sim 50k$ tokens, record MLP activations

h^{in}, h^{out} .

2. **Encode:** Use pretrained SAEs:

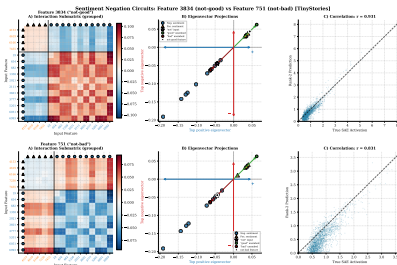
$$z = \text{TopK}(W_{\text{enc}}h)$$

3. **Build Q_f :** For output

feature f :

$$z_f^{\text{out}} \approx (z^{\text{in}})^T Q_f z^{\text{in}}$$

4. **Analyze:** Eigendecompose Q_f , visualize structure.



L1: SAE discovery ✓

SAEs find interpretable features ("not-good", "not-bad")

L2: Negation circuit ✓

AND-gate confirmed: negation $\otimes$ sentiment

L4: Sparse interact. ✓

Q_f shows clear block structure

Cosine similarity: Paper reports $\cos = -0.975$ (ts-medium, L4). We get $\cos = -0.16$ (fw-medium, L7) — weakly negative, not true opposites.

Model difference: Paper uses ts-medium (L4, 4x SAE). We use fw-medium (L7, 8x SAE) because ts-medium lacks `mlp-in` SAEs on HuggingFace.

Language Results: Claims L3, L5 (Figure 9)

What we measure (Figure 9):

L3: Low-rank emergence

Data: $\sim 1.57\text{M}$ tokens $\rightarrow$ record $\mathbf{z}^{\text{in}}, \mathbf{z}^{\text{out}}$ per token.

Filter: Only active features ($z_f^{\text{out}} > 0$).

Predict: $\hat{\mathbf{z}}_f^{\text{out}} = \sum_{r=1}^k \lambda_r (\mathbf{v}_r^\top \mathbf{z}^{\text{in}})^2$

Measure: Pearson corr $\Rightarrow$ high at $k=2$ = **low-rank**

L5: Cross-model test

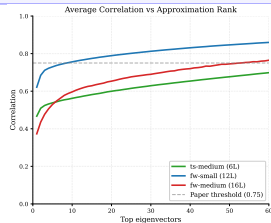
Paper: “most” features corr > 0.75

Ours:

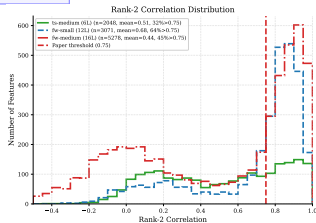
fw-small: 65% (198 tok/param) ✓

ts-medium: 32% (83 tok/param)

fw-medium: 45% (99 tok/param)



Correlation vs rank k



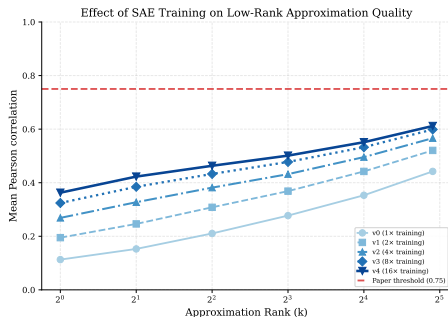
Rank-2 correlation histogram

Why vary?

Higher tok/param
 $\rightarrow$ better low-rank
behavior

L3 $\sim$ L5 $\sim$
(partial)

Language: Training Time Effect (Figure 10)



Hypothesis: Low-rank emergence requires sufficient training.

Evidence: SAE training progression (v0 $\rightarrow$ v4) shows:

- Fraction of features with corr > 0.75 increases **2.5 $\times$**
- More training $\rightarrow$ better low-rank structure

Tokens/param ratio:

- fw-small: 198 $\rightarrow$ 65% high corr
- fw-medium: 99 $\rightarrow$ 45%
- ts-medium: 83 $\rightarrow$ 32%

Models trained longer (higher tok/param) show stronger low-rank behavior.

Reproduction Summary

ID	Description	Status	Notes
<i>Vision (Section 4)</i>			
V1	Low-rank emergence	✓	EffRank: 15.5 vs. 38.5
V2	Interpretable eigenvectors	✓	Clear digit patterns
V3	Overfitting without reg.	✓	Diffuse patterns
V4	Regularization roles	✓	WD ↓ rank; noise ↑ acc
V5	Cross-seed consistency	✓	Top eigenvecs similar; acc low std
V6	Adversarial generation	✓	Eigenvector attacks work
<i>Language (Section 5)</i>			
L1	SAE feature discovery	✓	Pretrained SAEs work
L2	Negation circuits	✓	AND-gate confirmed
L3	Low-rank interactions	~	32–65% >0.75
L4	Sparse interactions	✓	Block structure found
L5	Cross-model generalization	~	Method works; varies

Extension 1: Cross-Dataset Robustness

Hypothesis: Eigenvector decomposition generalizes across datasets

Test Cases:

- ▶ **Writer Independence:** MNIST $\leftrightarrow$ EMNIST-Digits
- ▶ **Domain Transfer:** MNIST $\rightarrow$ USPS ($16 \times 16 \rightarrow 28 \times 28$)
- ▶ **Geometric Semantics:** MNIST $\rightarrow$ EMNIST-Letters
 - ▶ $O \rightarrow 0, I \rightarrow 1, Z \rightarrow 2, S \rightarrow 5$

Intuition: Geometric resemblance should lead to similar eigenvectors

All models use Center-of-Mass (CoM) normalization to focus on shape geometry

Extension 1: Results - Generalization Works!

Cross-Dataset Accuracy:

- ▶ MNIST $\rightarrow$ EMNIST-Digits: **97.7%**
- ▶ EMNIST-Digits $\rightarrow$ MNIST: **97.9%**

USPS Transfer:

- ▶ Regularized: **72.3%**
- ▶ Baseline: 53.4%
- ▶ Improvement: **+19 pp**

Letter $\rightarrow$ Digit Mapping:

- ▶ O $\rightarrow$ 0: **99.6%**
- ▶ I $\rightarrow$ 1: **87.1%**
- ▶ Z $\rightarrow$ 2: **88.8%**
- ▶ S $\rightarrow$ 5: **87.4%**

Extension 1: Cosine Similarity Limitations

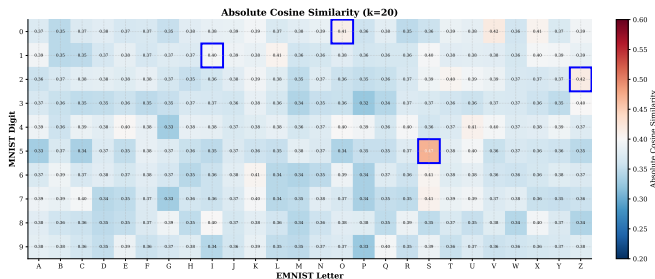


Figure: Absolute cosine similarity between MNIST digit classes and EMNIST letter classes fails to distinguish geometrically similar pairs (0–O) from dissimilar pairs (0–X).

Methodology: Absolute cosine similarity between the **first 20 eigenvectors** of each letter-digit pair, **averaged across 5 seeds**.

Cosine similarity fails: Similar pairs (0–O): 0.358 vs. Dissimilar pairs (0–X): 0.339 (virtually identical)

Quadratic Form Similarity

Problem: Cosine similarity fails

- ▶ Similar pairs (0–0): 0.358
- ▶ Dissimilar pairs (0–X): 0.339
- ▶ Not discriminative!

Solution: Quadratic Form Similarity

$$\text{sim}(A, B) = \frac{\text{tr}(A \cdot B)}{\|A\|_F \|B\|_F} \in [-1, 1]$$

Results:

- ▶ Similar pairs: **0.400**
- ▶ Dissimilar pairs: **-0.060**
- ▶ Large gap with $p < 10^{-4}$

Incorporates eigenvalue signs and eigenvector alignment

Extension 1: Quadratic Form Similarity Heatmap

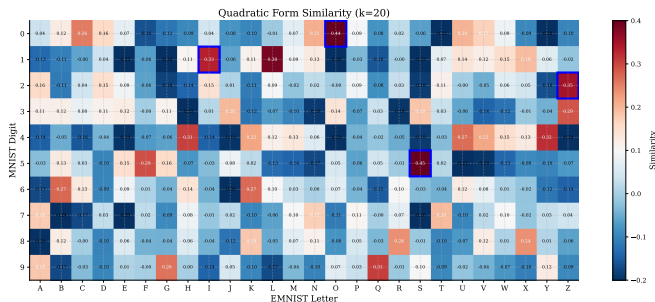


Figure: Quadratic Form Similarity Heatmap ($k=20$)

$k=20$: We take the rank-20 (low-rank) approximation of the interaction matrix B_c for each digit and each letter, then compute Quadratic Form Similarity.

Clear discrimination: strong diagonal for MNIST vs. EMNIST-Digits; bright entries at shape-matched pairs (0-O, 1-I, 2-Z, 5-S)

Extension 1: Key Figures

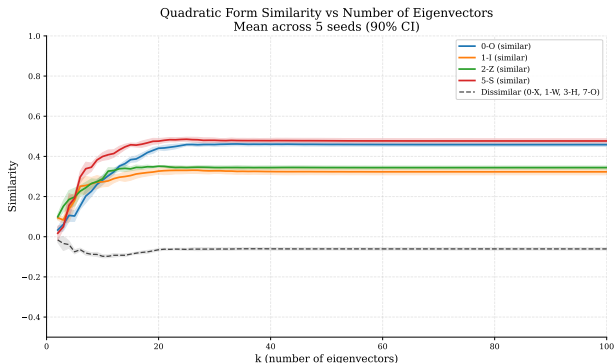


Figure: Quadratic Form Similarity vs number of eigenvectors k . Similar pairs converge around $k = 20$; dissimilar pairs stay near zero.

Main Takeaway: Quadratic Form Similarity is the correct metric for measuring structural similarity across models trained on different datasets—**not cosine similarity**.

Extension 2: CP-Decomposition

Motivation:

- ▶ Post-hoc → **intrinsic** interpretability
- ▶ Standard eigendecomposition: orthogonality → entangled
- ▶ CP: disentangled features

CP Factorization:

$$W_{cij} = \sum_{r=1}^R \lambda_r A_{cr} B_{ir} C_{jr}$$

Key Properties:

- ▶ No orthogonality constraint
- ▶ Disentangled representations
- ▶ Parameter reduction → faster training and scalability

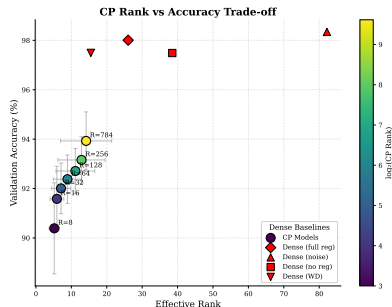


Figure: Accuracy vs rank trade-off

Results at $R = 256$:

- ▶ Accuracy: **93.8%**
- ▶ Effective rank: **17.5**
- ▶ vs Dense: 97.5% (3.7 pp drop)

CP-Decomposition: Efficiency

Experiment	Runs	Wall (h)	GPU-h	CO2 (kg)
Vision (Section 4)	101	1.66	1.11	0.040
Language (Section 5)	3	19.09	0.00	0.105
Extension 1	15	0.58	0.58	0.020
Extension 2 (CP)	105	0.05	0.05	0.004
Total	224	21.39	1.74	0.169

Table: Environmental impact (CO2 via codecarbon)

Key Efficiency Gains:

- ▶ **Parameter reduction:** $\mathcal{O}(d_{\text{hidden}} \times d_{\text{in}}^2) \rightarrow \mathcal{O}(R \times d_{\text{in}})$
- ▶ Average GPU-h per run: CP (0.00046) vs Vision* (0.014) — **30× more efficient**
- ▶ Only **3.7 pp accuracy drop** for significant parameter efficiency

*Based on 81 GPU-tracked vision experiments (excluding challenge task)

CP-Decomposition: Disentangled Factors



Figure: Feature disentanglement: Dense vs. CP

Dense Eigenvectors:

- ▶ Holistic, superimposed templates
- ▶ Entangled representations

CP Factors:

- ▶ Atomic, localized factors
- ▶ “Parts-based” representations

Thank You!

Questions?

Itay Erlich, May Ben Zion, Jacob Shashoua, Sagi Schwartz

FACT-AI Course Project, University of Amsterdam

CP-Decomposition Variants

Three variants with different regularization approaches

Fixed CP

Fixed CP: Hard rank constraint with no additional regularization

Characteristics:

- ▶ Strict complexity bound: rank R is fixed during training
- ▶ No regularization on the factors or scaling parameters
- ▶ Operates as a strict bottleneck

Results:

- ▶ Effective rank ~ 3.5 regardless of allocated capacity ($R \in \{32, 64, 128, 256\}$)
- ▶ Accuracy: $\sim 91.9\%$ (vs. 97.5% dense)
- ▶ $\sim 6\%$ accuracy cost for extreme sparsity

Forces extreme sparsity at the cost of accuracy

Lambda CP (L1 Regularization)

Lambda CP: L_1 regularization on learnable scales λ_r

Characteristics:

- ▶ Soft sparsity via L_1 regularization on λ_r parameters
- ▶ Allows factors to be pruned naturally during training
- ▶ More flexible than Fixed CP

Results:

- ▶ At $R = 256$: **93.8%** accuracy
- ▶ Effective rank: **17.5**
- ▶ Comparable to dense models with weight decay (eff. rank 15.5)

Provides robust middle ground: competitive accuracy with structural disentanglement

Gated CP (L0 Proxy)

Gated CP: Hard sigmoid gates with L_0 proxy for explicit rank pruning

Characteristics:

- ▶ Uses L_0 regularization proxy: $\sum_r \sigma(\phi_r)$ approximates count of active ranks
- ▶ Learnable gate logits ϕ_r control continuous gate values (0–1)
- ▶ Hard sigmoid gates: $\text{clamp}(\sigma(\phi_r) \cdot 1.1 - 0.05, 0, 1)$
- ▶ Most flexible regularization approach

Results:

- ▶ At $R = 256$: **93.79%** accuracy
- ▶ Effective rank: **17.35**
- ▶ Similar performance to Lambda CP

Alternative approach to Lambda CP with similar performance; both provide efficient, interpretable modeling